Pontificia Universidad Católica Madre y Maestra

Escuela de Ingeniería en Computación y Telecomunicaciones

ICC-223 · Bases de Datos

Segunda Prueba Escrita
Examen resuelto con solución integrada

Fecha del examen: 24 de marzo de 2026

Profesora	Lisibonny Beato
Material base utilizado	2do Examen Kelvin.pdf
Archivos de apoyo	restaurante_atencion.sql, diagrama de la base, diapositivas de SQL Parte I y Parte II
Propósito del documento	Presentar el enunciado y la solución explicada en un solo documento, listo para estudiar o entregar como referencia.

Nota importante. En los apartados 2.e y 2.f el enunciado indica que se sustituya “lb” por las iniciales del estudiante. En esta versión se mantiene “lb” como marcador para que puedas reemplazarlo por tus iniciales reales antes de ejecutar o entregar el script.

1. Presentación general

Este documento integra el examen y su solución completa. Se mantiene la estructura del enunciado original y, en cada inciso, se incluye una respuesta explicada, organizada y directamente vinculada con lo que se pidió en la prueba.

En la segunda parte se utiliza la base de datos RESTAURANTE_ATENCION suministrada en el archivo SQL, junto con el diagrama de referencia y el material de la unidad sobre lenguaje SQL. Las consultas fueron redactadas de forma clara y funcional, pensando tanto en el estudio como en la correcta ejecución en SQL Server.

2. Parte 1 · Desarrollo teórico

1.a) DML en SQL

Enunciado: Explique qué es el lenguaje DML (Data Manipulation Language) en SQL. Describa sus principales comandos y cuál es el propósito de cada uno dentro de una base de datos.

DML significa Data Manipulation Language. Es el conjunto de instrucciones de SQL que permite trabajar directamente con los datos almacenados en las tablas. Su función principal es consultar, insertar, modificar y eliminar registros.

Los comandos más importantes de DML son:

- SELECT: recupera datos de una o varias tablas.
- INSERT: agrega nuevos registros.
- UPDATE: modifica los valores de registros existentes.
- DELETE: elimina registros.

En conjunto, estos comandos permiten manipular la información de la base de datos sin cambiar la estructura de las tablas.

```
SELECT * FROM Clientes;

INSERT INTO Clientes (Nombre, Apellido)
VALUES ('Ana', 'Pérez');

UPDATE Clientes
SET Telefono = '809-555-0000'
WHERE ClienteID = 1;

DELETE FROM Clientes
WHERE ClienteID = 10;
```

1.b) PRIMARY KEY y FOREIGN KEY

Enunciado: Explique cómo se implementan las claves primarias y claves foráneas en SQL. Describa su sintaxis básica y cómo ayudan a garantizar la integridad referencial.

La PRIMARY KEY identifica de forma única cada fila de una tabla. No admite valores nulos y no puede repetirse. La FOREIGN KEY establece una relación entre dos tablas, obligando a que el valor almacenado exista previamente en la tabla referenciada.

Estas restricciones garantizan la integridad referencial porque evitan registros huérfanos o referencias a datos inexistentes.

```
CREATE TABLE Clientes (
   ClienteID INT NOT NULL PRIMARY KEY,
   Nombre VARCHAR(50) NOT NULL
);

CREATE TABLE Ordenes (
   OrdenID INT NOT NULL PRIMARY KEY,
   ClienteID INT NOT NULL,
   CONSTRAINT FK_Ordenes_Clientes
      FOREIGN KEY (ClienteID)
      REFERENCES Clientes(ClienteID)
);
```

En el ejemplo anterior, no se puede insertar una orden con un ClienteID que no exista en la tabla Clientes.

1.c) Plan de ejecución

Enunciado: Explique qué es un plan de ejecución (execution plan) en SQL, para qué se utiliza y cómo ayuda en la optimización de consultas. Mencione al menos dos elementos observables.

Un plan de ejecución es la representación de cómo el motor de base de datos decide ejecutar una consulta. Muestra el camino que seguirá para buscar, filtrar, unir y devolver la información solicitada.

Se utiliza para analizar el rendimiento de una consulta y detectar cuellos de botella. Es especialmente útil para optimizar consultas lentas, porque permite identificar operaciones costosas o ineficientes.

Entre los elementos que se pueden observar en un execution plan están los siguientes:

- Table Scan o Index Scan / Index Seek.
- Operadores de unión como Nested Loops, Merge Join o Hash Match.
- Costo estimado y cantidad de filas procesadas.

1.d) Índices

Enunciado: Describa qué es un índice en una base de datos. Explique sus principales ventajas y posibles desventajas en el rendimiento del sistema.

Un índice es una estructura auxiliar que acelera la localización de datos dentro de una tabla, de manera similar al índice de un libro. Su objetivo principal es reducir el tiempo de búsqueda en consultas.

Ventajas principales:

- Mejora la velocidad de búsquedas y filtros.
- Puede acelerar joins y ordenamientos.

Posibles desventajas:

- Consume espacio adicional en disco.
- Puede volver más lentas las operaciones INSERT, UPDATE y DELETE porque el índice también debe actualizarse.
- Demasiados índices pueden perjudicar el rendimiento general.

1.e) Vistas

Enunciado: Explique qué es una vista en SQL. Describa en qué se diferencia de una tabla y mencione dos ventajas de su uso.

Una vista es una tabla virtual construida a partir de una consulta SELECT. A diferencia de una tabla física, normalmente no almacena los datos de forma independiente, sino la definición de la consulta que los genera.

Dos ventajas importantes de las vistas son:

- Permiten simplificar consultas complejas y reutilizarlas.
- Ayudan a controlar el acceso a la información mostrando solo determinadas columnas o filas.

```
CREATE VIEW view_clientes AS
SELECT Nombre, Apellido, Email
FROM Clientes;
```

1.f) Control de acceso

Enunciado: Explique qué es el control de acceso en bases de datos. Describa cómo se implementa en SQL mediante usuarios, roles y permisos (GRANT, REVOKE), e indique por qué es importante para la seguridad de la información.

El control de acceso es el mecanismo que define qué usuarios pueden acceder a la base de datos y qué acciones están autorizados a realizar. Se implementa mediante usuarios, roles y permisos.

En SQL, los permisos se administran con instrucciones como GRANT para conceder privilegios y REVOKE para retirarlos.

```
GRANT SELECT, INSERT ON Clientes TO usuario1;
REVOKE INSERT ON Clientes FROM usuario1;
```

Esto es importante porque protege la información, evita modificaciones no autorizadas y mejora la seguridad general del sistema.

3. Parte 2 · Base de datos RESTAURANTE_ATENCION

Enunciado general: crear la base de datos localmente a partir del archivo SQL y, con base en su estructura y datos, elaborar las consultas y objetos solicitados en los incisos a-f.

La base suministrada contiene las tablas Meseros, Mesas, Clientes, Menu, Ordenes y DetalleOrden. La relación central está en Ordenes, que conecta con clientes, mesas, meseros y los detalles consumidos en el menú.

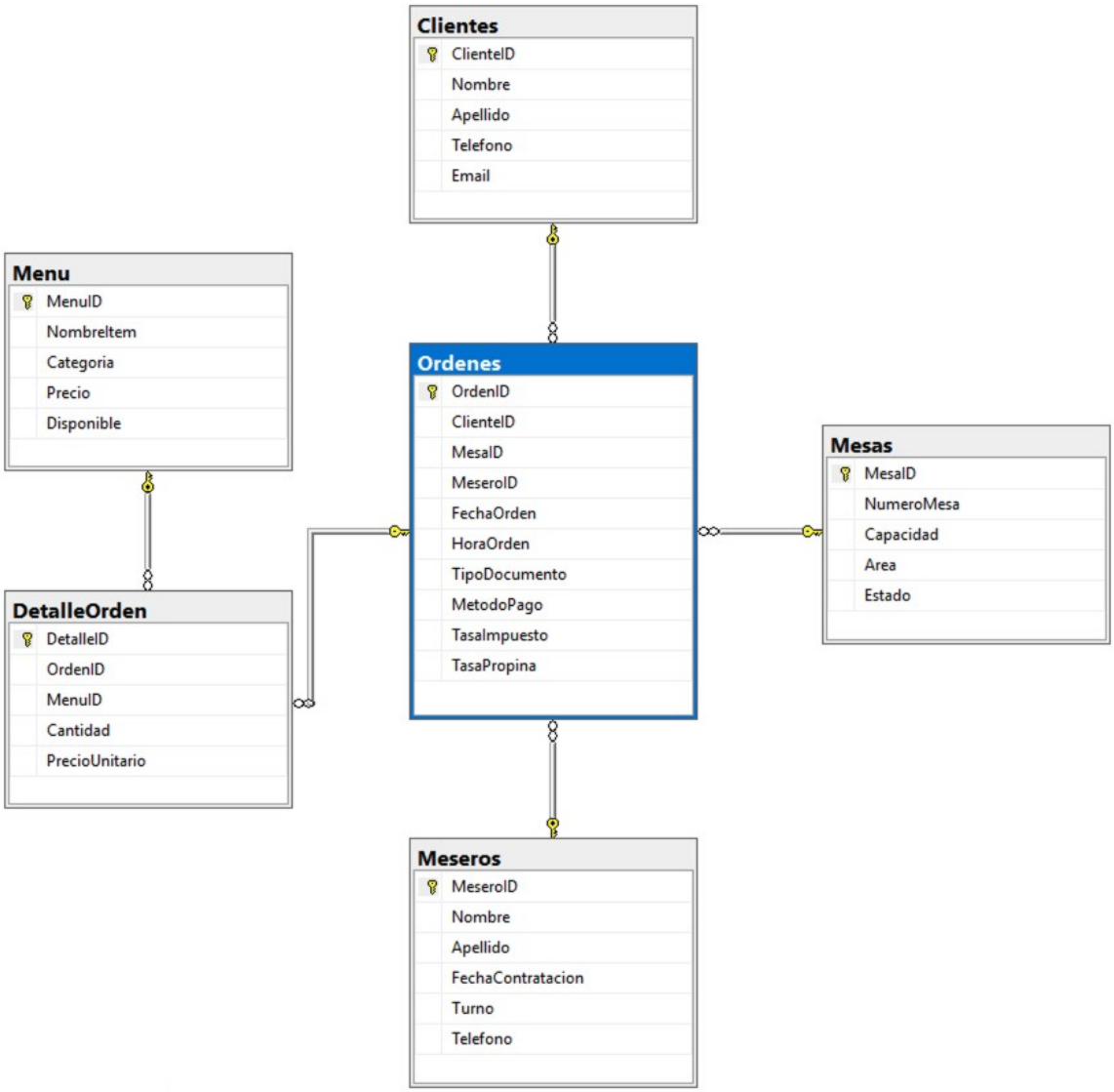


Figura 1. Diagrama de referencia de la base de datos RESTAURANTE_ATENCION.

Nota importante. Para el inciso 2.b se siguió exactamente la instrucción del examen: al subtotal de cada orden se le añade directamente el valor de TasaImpuesto. No se interpretó como porcentaje, sino como el monto tal como está redactado el enunciado.

2.a) Meseros con propina mayor a 10

Objetivo: obtener el nombre y apellido de los meseros que atendieron órdenes con tasa de propina superior a 10, sin repetir filas.

```
SELECT DISTINCT
   m.Nombre,
   m.Apellido
```

```
FROM dbo.Meseros AS m
INNER JOIN dbo.Ordenes AS o
   ON m.MeseroID = o.MeseroID
WHERE o.TasaPropina > 10;
```

La consulta enlaza Meseros con Ordenes por MeseroID, filtra las órdenes cuya propina supera 10 y usa DISTINCT para evitar duplicados.

Resultado esperado:

Nombre	Apellido
Carlos	Gómez
María	Fernández
José	Ramírez

2.b) Suma total pagada en órdenes con al menos un postre

Objetivo: calcular la suma total pagada en las órdenes donde se consumió al menos un producto de categoría Postre. El total pagado por orden es subtotal + TasaImpuesto.

```
SELECT
   SUM(x.Subtotal + x.TasaImpuesto) AS SumaTotalPagada
FROM (
   SELECT
      o.OrdenID,
      SUM(d.Cantidad * d.PrecioUnitario) AS Subtotal,
      o.TasaImpuesto
   FROM dbo.Ordenes AS o
   INNER JOIN dbo.DetalleOrden AS d
      ON o.OrdenID = d.OrdenID
   WHERE EXISTS (
      SELECT 1
      FROM dbo.DetalleOrden AS d2
      INNER JOIN dbo.Menu AS m2
         ON d2.MenuID = m2.MenuID
      WHERE d2.OrdenID = o.OrdenID
        AND m2.Categoria = 'Postre'
   )
   GROUP BY o.OrdenID, o.TasaImpuesto
) AS x;
```

Primero se localizan las órdenes que contienen al menos un postre. Luego se calcula el subtotal de cada una y, finalmente, se suma el impuesto indicado por el enunciado.

Órdenes involucradas: 2, 4, 6 y 8.

OrdenID	Subtotal	TasaImpuesto	Total considerado
2	15.75	18.00	33.75
4	49.00	18.00	67.00
6	24.75	18.00	42.75
8	32.00	18.00	50.00

Suma total pagada: 193.50

2.c) Clientes con más de una orden y subtotal superior a 20

Objetivo: listar el nombre y apellido del cliente junto con la fecha de sus órdenes, considerando solo clientes con más de una orden y órdenes cuyo subtotal supere 20. Debe ordenarse por nombre ascendente y fecha descendente.

```
WITH Subtotales AS (
    SELECT
        o.OrdenID,
        o.ClienteID,
        o.FechaOrden,
        SUM(d.Cantidad * d.PrecioUnitario) AS Subtotal
    FROM dbo.Ordenes AS o
    INNER JOIN dbo.DetalleOrden AS d
        ON o.OrdenID = d.OrdenID
    GROUP BY o.OrdenID, o.ClienteID, o.FechaOrden
)
SELECT
    c.Nombre,
    c.Apellido,
    s.FechaOrden
FROM Subtotales AS s
INNER JOIN dbo.Clientes AS c
    ON s.ClienteID = c.ClienteID
WHERE s.Subtotal > 20
  AND s.ClienteID IN (
      SELECT ClienteID
      FROM dbo.Ordenes
      GROUP BY ClienteID
      HAVING COUNT(*) > 1
  )
ORDER BY c.Nombre ASC, s.FechaOrden DESC;
```

La CTE calcula el subtotal de cada orden. Después se filtran los clientes con más de una orden y se conservan únicamente las órdenes cuyo subtotal es mayor que 20.

Nombre	Apellido	FechaOrden
Andrés	Martínez	2026-02-15
Laura	Pérez	2026-02-13
Laura	Pérez	2026-02-10
Sofía	López	2026-02-16

2.d) Ventas por producto entre el 10 y el 15 de febrero de 2026

Objetivo: listar la categoría, el nombre del producto, la cantidad total de unidades vendidas y el total vendido por producto en el rango de fechas indicado.

```
SELECT
    m.Categoria,
    m.NombreItem,
    SUM(d.Cantidad) AS CantidadTotalVendida,
    SUM(d.Cantidad * d.PrecioUnitario) AS TotalVendido
FROM dbo.Ordenes AS o
INNER JOIN dbo.DetalleOrden AS d
```

```
   ON o.OrdenID = d.OrdenID
INNER JOIN dbo.Menu AS m
   ON d.MenuID = m.MenuID
WHERE o.FechaOrden BETWEEN '2026-02-10' AND '2026-02-15'
GROUP BY
   m.Categoria,
   m.NombreItem
ORDER BY
   m.Categoria,
   m.NombreItem;
```

La consulta agrupa por categoría y producto, suma las cantidades vendidas y calcula el total monetario multiplicando cantidad por precio unitario.

Categoría	Producto	Cant. total	Total vendido
Bebida	Café	3	6.00
Bebida	Jugo Natural	5	15.00
Bebida	Refresco	7	17.50
Entrada	Croquetas de Queso	2	11.50
Entrada	Ensalada César	1	6.50
Plato Fuerte	Hamburguesa Clásica	5	47.50
Plato Fuerte	Pasta Alfredo	1	11.00
Plato Fuerte	Salmón a la Plancha	3	49.50
Postre	Flan de Caramelo	4	17.00

2.e) Vista del promedio de propina por tipo de documento

Objetivo: crear una vista que muestre el porcentaje promedio de propina aplicado en las órdenes, agrupado por tipo de documento.

```
CREATE VIEW dbo.view_e_lb
AS
SELECT
   TipoDocumento,
   AVG(CAST(TasaPropina AS DECIMAL(10,2))) AS PorcentajePromedioPropina
FROM dbo.Ordenes
GROUP BY TipoDocumento;
```

Esta vista resume la propina promedio según el tipo de documento emitido en la orden.

TipoDocumento	Promedio de propina
Factura	11.80
Ticket	8.67

2.f) Función escalar para promedio de propina por tipo de documento

Objetivo: crear una función que reciba un tipo de documento y devuelva la tasa de propina promedio de las órdenes de ese tipo.

```
CREATE FUNCTION dbo.fn_f_lb (@TipoDocumento VARCHAR(20))
RETURNS DECIMAL(10,2)
AS
BEGIN
   DECLARE @Promedio DECIMAL(10,2);

   SELECT
```

```
    @Promedio = AVG(CAST(TasaPropina AS DECIMAL(10,2)))
  FROM dbo.Ordenes
  WHERE TipoDocumento = @TipoDocumento;

  RETURN ISNULL(@Promedio, 0);
END;
GO

SELECT dbo.fn_f_lb('Factura') AS PromedioFactura,
    dbo.fn_f_lb('Ticket')  AS PromedioTicket;
```

La función permite consultar el promedio de propina de forma reutilizable, pasando el tipo de documento como argumento.

Prueba	Resultado
dbo.fn_f_lb('Factura')	11.80
dbo.fn_f_lb('Ticket')	8.67

4. Conclusión

La solución presentada cumple con los requerimientos del examen en sus dos partes: desarrolla los conceptos teóricos fundamentales del lenguaje SQL y resuelve las consultas y objetos pedidos sobre la base RESTAURANTE_ATENCION.

Como recomendación final, antes de entregar el script en un entorno académico, conviene sustituir el marcador “lb” por las iniciales reales del estudiante y ejecutar cada inciso en SQL Server para verificar que el resultado mostrado coincida con el esperado.

Anexo · Script consolidado de la parte 2

A continuación se incluye un bloque continuo con las consultas y objetos de la segunda parte. Este anexo es útil para copiar y ejecutar directamente en SQL Server.

```
USE RESTAURANTE_ATENCION;
GO

-- a)
SELECT DISTINCT
  m.Nombre,
  m.Apellido
FROM dbo.Meseros AS m
INNER JOIN dbo.Ordenes AS o
  ON m.MeseroID = o.MeseroID
WHERE o.TasaPropina > 10;
GO

-- b)
SELECT
  SUM(x.Subtotal + x.TasaImpuesto) AS SumaTotalPagada
FROM (
  SELECT
    o.OrdenID,
    SUM(d.Cantidad * d.PrecioUnitario) AS Subtotal,
```

```
        o.TasaImpuesto
    FROM dbo.Ordenes AS o
    INNER JOIN dbo.DetalleOrden AS d
        ON o.OrdenID = d.OrdenID
    WHERE EXISTS (
        SELECT 1
        FROM dbo.DetalleOrden AS d2
        INNER JOIN dbo.Menu AS m2
            ON d2.MenuID = m2.MenuID
        WHERE d2.OrdenID = o.OrdenID
          AND m2.Categoria = 'Postre'
    )
    GROUP BY o.OrdenID, o.TasaImpuesto
) AS x;
GO

-- c)
WITH Subtotales AS (
    SELECT
        o.OrdenID,
        o.ClienteID,
        o.FechaOrden,
        SUM(d.Cantidad * d.PrecioUnitario) AS Subtotal
    FROM dbo.Ordenes AS o
    INNER JOIN dbo.DetalleOrden AS d
        ON o.OrdenID = d.OrdenID
    GROUP BY o.OrdenID, o.ClienteID, o.FechaOrden
)
SELECT
    c.Nombre,
    c.Apellido,
    s.FechaOrden
FROM Subtotales AS s
INNER JOIN dbo.Clientes AS c
    ON s.ClienteID = c.ClienteID
WHERE s.Subtotal > 20
  AND s.ClienteID IN (
      SELECT ClienteID
      FROM dbo.Ordenes
      GROUP BY ClienteID
      HAVING COUNT(*) > 1
  )
ORDER BY c.Nombre ASC, s.FechaOrden DESC;
GO

-- d)
SELECT
    m.Categoria,
    m.NombreItem,
    SUM(d.Cantidad) AS CantidadTotalVendida,
    SUM(d.Cantidad * d.PrecioUnitario) AS TotalVendido
FROM dbo.Ordenes AS o
INNER JOIN dbo.DetalleOrden AS d
```

```
    ON o.OrdenID = d.OrdenID
INNER JOIN dbo.Menu AS m
    ON d.MenuID = m.MenuID
WHERE o.FechaOrden BETWEEN '2026-02-10' AND '2026-02-15'
GROUP BY
    m.Categoria,
    m.NombreItem
ORDER BY
    m.Categoria,
    m.NombreItem;
GO

-- e) Reemplazar lb por tus iniciales reales
CREATE VIEW dbo.view_e_lb
AS
SELECT
    TipoDocumento,
    AVG(CAST(TasaPropina AS DECIMAL(10,2))) AS PorcentajePromedioPropina
FROM dbo.Ordenes
GROUP BY TipoDocumento;
GO

-- f) Reemplazar lb por tus iniciales reales
CREATE FUNCTION dbo.fn_f_lb (@TipoDocumento VARCHAR(20))
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @Promedio DECIMAL(10,2);

    SELECT
        @Promedio = AVG(CAST(TasaPropina AS DECIMAL(10,2)))
    FROM dbo.Ordenes
    WHERE TipoDocumento = @TipoDocumento;

    RETURN ISNULL(@Promedio, 0);
END;
GO
```